

ESCUELA POLITÉCNICA NACIONAL

Facultad de Sistemas

Ingeniería en Ciencias de la Computación

Nombre del profesor: Mg . Marco Aguirre

Nombre: Justin Cusquillo

Curso: GR1CC

Fecha de entrega: 02 de noviembre del 2026

Tarea: creación de carpetas mediante procesos

“E SCIENTIA HOMINIS SALUS”

1. Introducción

En Linux, un programa puede crear nuevos procesos para repartir su trabajo, esta práctica usa os.fork() para que un proceso hijo cree carpetas con el comando mkdir, mientras el padre espera su finalización con os.waitpid(). El hijo ejecuta el comando de dos formas, con subprocess.run() y con os.system(), para comparar cómo funciona cada una, cómo controlan los errores y qué riesgos de seguridad aparecen al usar texto ingresado por el usuario

2. Objetivos

Aplicar os.fork() y os.waitpid() para crear y sincronizar un proceso padre y un proceso hijo en Linux
Comparar subprocess.run() y os.system() al ejecutar mkdir, evaluando su manejo de errores y su seguridad

3. Actividades de experimentación:

Código versión 1:

```
  GNU nano 8.7.1                                                    05_fork_sub
"""Versión 1: el proceso hijo crea las carpetas con subprocess.run()."""
import os
import re
import subprocess
import sys

DIRECTORIO_BASE = "carpetas_actividad_5"
PATRON_VALIDO = re.compile(r"^[A-Za-z0-9_-]+$")  # solo letras, números, - y _


def pedir_nombres():
    """PADRE: solicita y valida los nombres de las carpetas."""
    entrada = input("Ingrese nombres de carpetas separados por comas:\n")
    candidatos = [n.strip() for n in entrada.split(",") if n.strip()]

    validos, vistos = [], set()
    for nombre in candidatos:
        if not PATRON_VALIDO.match(nombre):
            print(f"[PADRE] Nombre rechazado (caracteres no permitidos): '{nombre}'")
        elif nombre in vistos:
            print(f"[PADRE] Nombre repetido en la entrada, se ignora: '{nombre}'")
        else:
            vistos.add(nombre)
            validos.append(nombre)
    return validos


def trabajo_del_hijo(nombres):
    """HIJO: ejecuta mkdir con subprocess.run() para cada carpeta."""
    print(f"[HIJO] Inicio del trabajo | PID hijo: {os.getpid()} | PPID: {os.getppid()}",
          flush=True)
    errores = 0
    for nombre in nombres:
        ruta = os.path.join(DIRECTORIO_BASE, nombre)
        if os.path.exists(ruta):
            print(f"[HIJO] Error: la carpeta '{nombre}' ya existe (no se duplica).")
            errores += 1
            continue
        # Lista de argumentos: NO se usa shell, el nombre nunca se interpreta.
        resultado = subprocess.run(["mkdir", "--", ruta],
                                   capture_output=True, text=True)
        if resultado.returncode == 0:
            print(f"[HIJO] Carpeta creada: {ruta}")
        else:
            print(f"[HIJO] Error al crear '{nombre}': {resultado.stderr.strip()}")
            errores += 1
    print(f"[HIJO] Fin del trabajo | errores: {errores}", flush=True)

def main():
    nombres = pedir_nombres()
    if len(nombres) < 5:
        print(f"[PADRE] Se requieren al menos 5 nombres válidos y distintos "
              f"(recibidos: {len(nombres)}).")
        sys.exit(1)

    os.makedirs(DIRECTORIO_BASE, exist_ok=True)
    print(f"[PADRE] PID del padre: {os.getpid()}", flush=True)

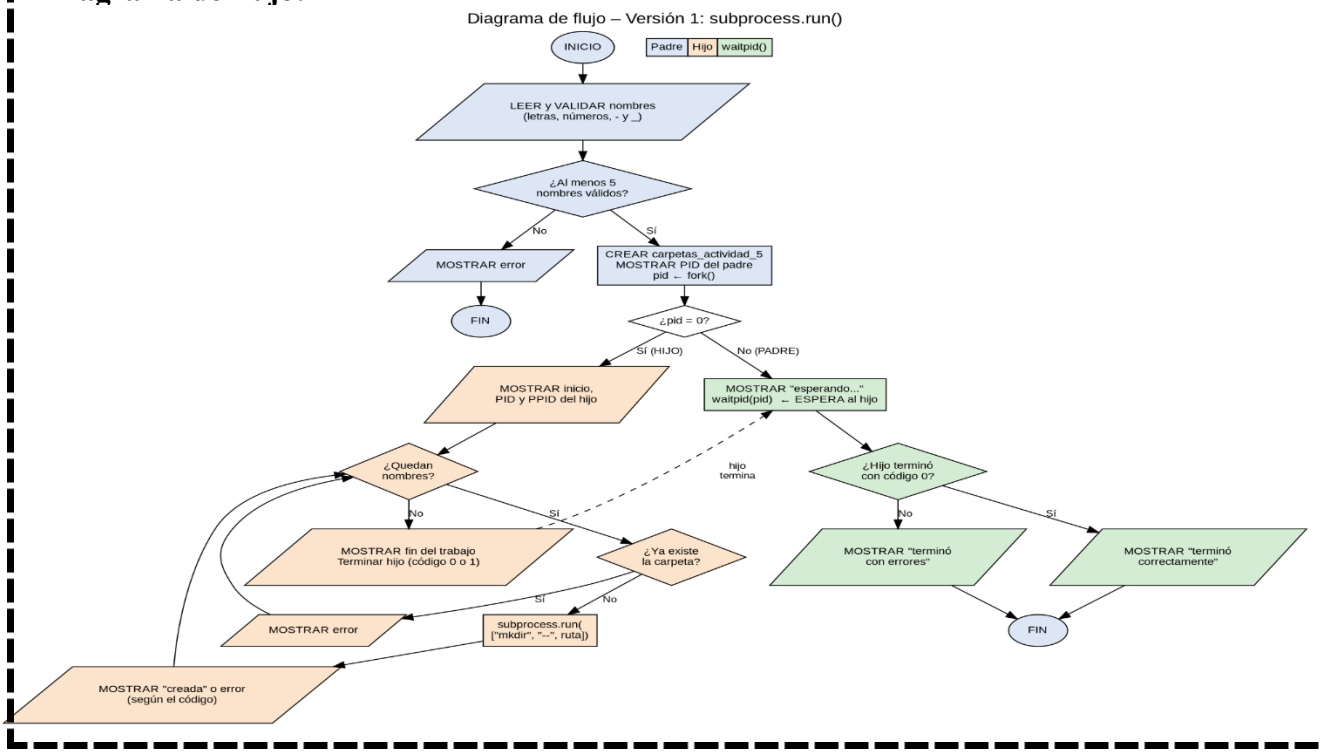
    pid = os.fork()
    if pid == 0:
        # ---- Código que solo ejecuta el HIJO ----
        codigo = 1
        try:
            codigo = trabajo_del_hijo(nombres)
        finally:
            os._exit(codigo)  # el hijo termina aquí y no sigue el código del padre
    else:
        # ---- Código que solo ejecuta el PADRE ----
        print(f"[PADRE] Hijo creado con PID {pid}. Esperando con waitpid()...",
              flush=True)
        _, estado = os.waitpid(pid, 0)
        if os.WIFEXITED(estado) and os.WEXITSTATUS(estado) == 0:
            print("[PADRE] El hijo terminó correctamente (código 0).")
        elif os.WIFEXITED(estado):
            print(f"[PADRE] El hijo terminó con errores (código {os.WEXITSTATUS(estado)}).")
        else:
            print("[PADRE] El hijo terminó de forma anormal (señal).")


if __name__ == "__main__":
    main()
```

Pseudocódigo:

```
INICIO
  LEER nombres separados por comas
  VALIDAR nombres (solo letras, números, - y _; sin repetidos)
  SI hay menos de 5 nombres válidos ENTONCES
     MOSTRAR error
     TERMINAR
  FIN SI
  CREAR carpetas_actividad_5 si no existe
  MOSTRAR PID
  pid ← fork()
  SI pid = 0 ENTONCES
     MOSTRAR inicio, PID y PPID
     PARA cada nombre HACER
        SI la carpeta ya existe ENTONCES
           MOSTRAR error
        SINO
           subprocess.run(["mkdir", "--", ruta])
           SI código = 0 ENTONCES
              MOSTRAR "carpeta creada"
           SINO
              MOSTRAR error
           FIN SI
        FIN SI
     FIN PARA
     MOSTRAR fin del trabajo
  SINO
     MOSTRAR "esperando..."
     waitpid(pid)
     SI terminó con código 0 ENTONCES
        MOSTRAR "terminó correctamente"
     SINO
        MOSTRAR "terminó con errores"
     FIN SI
  FIN SI
FIN
```

Diagrama de flujo:



Código versión 2:

```
  GNU nano 8.7.1                                                    05_fork_system.py *
"""Versión 2: el proceso hijo crea las carpetas con os.system()."""
import os
import re
import shlex
import sys

DIRECTORIO_BASE = "carpetas_actividad_5"
PATRON_VALIDO = re.compile(r"^[A-Za-z0-9_-]+$")  # solo letras, números, - y _


def pedir_nombres():
    """PADRE: solicita y valida los nombres de las carpetas."""
    entrada = input("Ingrese nombres de carpetas separados por comas:\n")
    candidatos = [n.strip() for n in entrada.split(",") if n.strip()]

    validos, vistos = [], set()
    for nombre in candidatos:
        if not PATRON_VALIDO.match(nombre):
            print(f"[PADRE] Nombre rechazado (caracteres no permitidos): '{nombre}'")
        elif nombre in vistos:
            print(f"[PADRE] Nombre repetido en la entrada, se ignora: '{nombre}'")
        else:
            vistos.add(nombre)
            validos.append(nombre)
    return validos


def trabajo_del_hijo(nombres):
    """HIJO: ejecuta mkdir con os.system() para cada carpeta."""
    print(f"[HIJO] Inicio del trabajo | PID hijo: {os.getpid()} | PPID: {os.getppid()}",
          flush=True)
    errores = 0
    for nombre in nombres:
        ruta = os.path.join(DIRECTORIO_BASE, nombre)
        if os.path.exists(ruta):
            print(f"[HIJO] Error: la carpeta '{nombre}' ya existe (no se duplica).")
            errores += 1
            continue
        # os.system() usa una shell: se escapa el texto con shlex.quote().
        estado = os.system(f"mkdir -- {shlex.quote(ruta)}")
        codigo = os.waitstatus_to_exitcode(estado)  # os.system devuelve estado tipo wait
        if codigo == 0:
            print(f"[HIJO] Carpeta creada: {ruta}")
        else:
            print(f"[HIJO] Error al crear '{nombre}' (código de salida {codigo}).")
            errores += 1
    print(f"[HIJO] Fin del trabajo | errores: {errores}", flush=True)


def main():
    nombres = pedir_nombres()
    if len(nombres) < 5:
        print(f"[PADRE] Se requieren al menos 5 nombres válidos y distintos "
              f"(recibidos: {len(nombres)}).")
        sys.exit(1)

    os.makedirs(DIRECTORIO_BASE, exist_ok=True)
    print(f"[PADRE] PID del padre: {os.getpid()}", flush=True)

    pid = os.fork()
    if pid == 0:
        # ---- Código que solo ejecuta el HIJO ----
        codigo = 1
        try:
            codigo = trabajo_del_hijo(nombres)
        finally:
            os._exit(codigo)  # el hijo termina aquí y no sigue el código del padre
    else:
        # ---- Código que solo ejecuta el PADRE ----
        print(f"[PADRE] Hijo creado con PID {pid}. Esperando con waitpid()...",
              flush=True)
        _, estado = os.waitpid(pid, 0)
        if os.WIFEXITED(estado) and os.WEXITSTATUS(estado) == 0:
            print("[PADRE] El hijo terminó correctamente (código 0).")
        elif os.WIFEXITED(estado):
            print(f"[PADRE] El hijo terminó con errores (código {os.WEXITSTATUS(estado)}).")
        else:
            print("[PADRE] El hijo terminó de forma anormal (señal).")

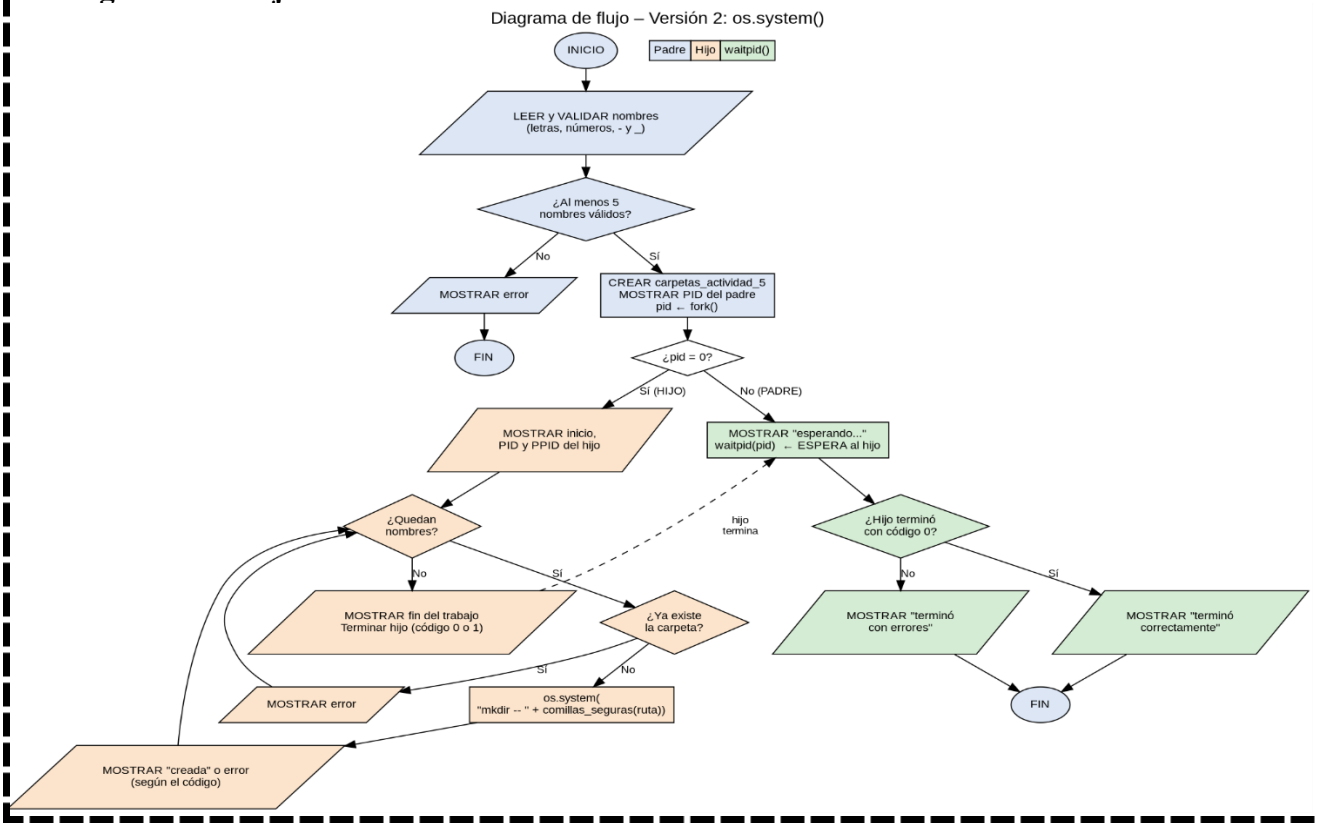

if __name__ == "__main__":
    main()
```

Pseudocódigo:

```
INICIO
 LEER nombres separados por comas
 VALIDAR nombres (solo letras, números, - y _; sin repetidos)
 SI hay menos de 5 nombres válidos ENTONCES
    MOSTRAR error
    TERMINAR
 FIN SI
 CREAR carpetas_actividad_5 si no existe
 MOSTRAR PID
 pid ← fork()

 SI pid = 0 ENTONCES
    MOSTRAR inicio, PID y PPID
    PARA cada nombre HACER
       SI la carpeta ya existe ENTONCES
          MOSTRAR error
       SINO
          estado ← os.system("mkdir -- " + comillas_seguras(ruta))
          SI código de salida = 0 ENTONCES
             MOSTRAR "carpeta creada"
          SINO
             MOSTRAR error
          FIN SI
       FIN SI
    FIN PARA
    MOSTRAR fin del trabajo
 SINO
    MOSTRAR "esperando..."
    waitpid(pid)
    SI terminó con código 0 ENTONCES
       MOSTRAR "terminó correctamente"
    SINO
       MOSTRAR "terminó con errores"
    FIN SI
 FIN SI
FIN
```

Diagrama de flujo:



1. Ejecute ambas versiones con al menos cinco carpetas. Verifique que los resultados sean equivalentes.

```
justin@ubuntu-server:~/tarea_procesos$ python3 05_fork_subprocess.py
Ingrese nombres de carpetas separados por comas:
Documentos, Imagenes, Proyectos, Respaldos, Temporal
[PADRE] PID del padre: 9177
[PADRE] Hijo creado con PID 9178. Esperando con waitpid()...
[HIJO] Inicio del trabajo | PID hijo: 9178 | PPID: 9177
[HIJO] Carpeta creada: carpetas_actividad_5/Documentos
[HIJO] Carpeta creada: carpetas_actividad_5/Imagenes
[HIJO] Carpeta creada: carpetas_actividad_5/Proyectos
[HIJO] Carpeta creada: carpetas_actividad_5/Respaldos
[HIJO] Carpeta creada: carpetas_actividad_5/Temporal
[HIJO] Fin del trabajo | errores: 0
[PADRE] El hijo terminó correctamente (código 0).
```

```
justin@ubuntu-server:~/tarea_procesos$ python3 05_fork_system.py
Ingrese nombres de carpetas separados por comas:
Documentos, Imagenes, Proyectos, Respaldos, Temporal
[PADRE] PID del padre: 9346
[PADRE] Hijo creado con PID 9347. Esperando con waitpid()...
[HIJO] Inicio del trabajo | PID hijo: 9347 | PPID: 9346
[HIJO] Carpeta creada: carpetas_actividad_5/Documentos
[HIJO] Carpeta creada: carpetas_actividad_5/Imagenes
[HIJO] Carpeta creada: carpetas_actividad_5/Proyectos
[HIJO] Carpeta creada: carpetas_actividad_5/Respaldos
[HIJO] Carpeta creada: carpetas_actividad_5/Temporal
[HIJO] Fin del trabajo | errores: 0
[PADRE] El hijo terminó correctamente (código 0).
```

```
justin@ubuntu-server:~/tarea_procesos$ diff v1.txt v2.txt && echo "Resultados Equivalentes"
Resultados Equivalentes
justin@ubuntu-server:~/tarea_procesos$
```

2. Identifique el PID del proceso padre, el PID del hijo y el PPID observado por el hijo.

Version1 :

```
[PADRE] PID del padre: 9177
[PADRE] Hijo creado con PID 9178. Esperando con waitpid()...
[HIJO] Inicio del trabajo | PID hijo: 9178 | PPID: 9177
```

Version 2 :

```
[PADRE] PID del padre: 9346
[PADRE] Hijo creado con PID 9347. Esperando con waitpid()...
[HIJO] Inicio del trabajo | PID hijo: 9347 | PPID: 9346
```

3. Determine qué parte del programa solicita los nombres de las carpetas, cuál ejecuta mkdir y cuál espera la finalización.

```
1:"""Versión 1: el proceso hijo crea las carpetas con subprocess.run()."""
11:def pedir_nombres():
13:    entrada = input("Ingrese nombres de carpetas separados por comas:\n")
29:    """HIJO: ejecuta mkdir con subprocess.run() para cada carpeta."""
40:        resultado = subprocess.run(["mkdir", "--", ruta],
61:    pid = os.fork()
73:        _, estado = os.waitpid(pid, 0)
```

```
1:"""Versión 2: el proceso hijo crea las carpetas con os.system()."""
11:def pedir_nombres():
13:    entrada = input("Ingrese nombres de carpetas separados por comas:\n")
29:    """HIJO: ejecuta mkdir con os.system() para cada carpeta."""
39:        # os.system() usa una shell: se escapa el texto con shlex.quote().
40:        estado = os.system(f"mkdir -- {shlex.quote(ruta)}")
41:        codigo = os.waitstatus_to_exitcode(estado)  # os.system devuelve estado tipo wait
61:    pid = os.fork()
73:        _, estado = os.waitpid(pid, 0)
```

4. Repita la ejecución utilizando los mismos nombres. Registre el comportamiento y los mensajes de error, si existen.

```
justin@ubuntu-server:~/tarea_procesos$ python3 05_fork_subprocess.py
Ingrese nombres de carpetas separados por comas:
Documentos, Imagenes, Proyectos, Respaldos, Temporal
[PADRE] PID del padre: 9507
[PADRE] Hijo creado con PID 9508. Esperando con waitpid()...
[HIJO] Inicio del trabajo | PID hijo: 9508 | PPID: 9507
[HIJO] Error: la carpeta 'Documentos' ya existe (no se duplica).
[HIJO] Error: la carpeta 'Imagenes' ya existe (no se duplica).
[HIJO] Error: la carpeta 'Proyectos' ya existe (no se duplica).
[HIJO] Error: la carpeta 'Respaldos' ya existe (no se duplica).
[HIJO] Error: la carpeta 'Temporal' ya existe (no se duplica).
[HIJO] Fin del trabajo | errores: 5
[PADRE] El hijo terminó con errores (código 1).
justin@ubuntu-server:~/tarea_procesos$
justin@ubuntu-server:~/tarea_procesos$ python3 05_fork_system.py
Ingrese nombres de carpetas separados por comas:
Documentos, Imagenes, Proyectos, Respaldos, Temporal
[PADRE] PID del padre: 9529
[PADRE] Hijo creado con PID 9530. Esperando con waitpid()...
[HIJO] Inicio del trabajo | PID hijo: 9530 | PPID: 9529
[HIJO] Error: la carpeta 'Documentos' ya existe (no se duplica).
[HIJO] Error: la carpeta 'Imagenes' ya existe (no se duplica).
[HIJO] Error: la carpeta 'Proyectos' ya existe (no se duplica).
[HIJO] Error: la carpeta 'Respaldos' ya existe (no se duplica).
[HIJO] Error: la carpeta 'Temporal' ya existe (no se duplica).
[HIJO] Fin del trabajo | errores: 5
[PADRE] El hijo terminó con errores (código 1).
justin@ubuntu-server:~/tarea_procesos$
```

Actividad de consulta Investigue y discuta las diferencias entre subprocess.run() y os.system(), considerando la ejecución de comandos, el manejo de argumentos, el control de errores, el estado de salida y el uso de una shell. Explique también por qué la construcción de comandos a partir de texto ingresado por el usuario requiere precauciones de seguridad

Aspecto	subprocess.run()	os.system()
Ejecución del comando	Crea un proceso que ejecuta el programa directamente	Ejecuta /bin/sh -c con una cadena de texto
Argumentos	Lista (["mkdir", "--", ruta]); cada elemento es un argumento literal	Una sola cadena que la shell interpreta
Control de errores	check=True, timeout, excepciones y captura de stderr	Solo devuelve un número entero

Estado de salida	resultado.returncode	Estado codificado; se decodifica con os.waitstatus_to_exitcode()
Uso de shell	No, salvo que se indique shell=True	Siempre

Seguridad con texto ingresado por el usuario
La shell interpreta caracteres especiales como ;, &&, |, $() y *. Si se arma un comando con texto del usuario, este puede inyectar comandos. Por ejemplo, con os.system("mkdir " + nombre), si el usuario escribe Docs; rm -rf ~, se ejecutan dos comandos y el segunda borra su directorio personal. A esto se le llama inyección de comandos
Precauciones aplicadas en la tarea:

- Validar con lista blanca
- Usar subprocess.run() con lista de argumentos, sin shell
- Usar shlex.quote() cuando se emplea os.system()
- Usar -- para que un nombre que empiece con - no se tome como opción

4. Tabla Comparativa

Criterio	subprocess.run()	os.system()
Ejecución del comando	Crea un proceso que ejecuta mkdir directamente	Lanza /bin/sh -c con la cadena y la shell ejecuta mkdir
Uso de intérprete de comandos	No (shell=False por defecto)	Siempre usa shell (/bin/sh)
Manejo de argumentos	Lista ["mkdir", "--", ruta]; cada elemento es un argumento literal	Una sola cadena "mkdir -- ruta" que la shell separa e interpreta
Control de errores	returncode, stderr capturado, check=True, timeout y excepciones	Solo devuelve un entero; el error de mkdir se imprime en la terminal y no se captura
Estado de finalización	resultado.returncode (0 = éxito)	Estado codificado; se obtiene el código con os.waitstatus_to_exitcode()
Seguridad de la entrada	Segura al pasar el texto como argumento, sin shell	Riesgo de inyección de comandos; requiere validar y usar shlex.quote()

5. Preguntas de análisis
1. ¿Cuál es la diferencia entre el proceso hijo creado por os.fork() y los procesos que pueden ser iniciados por subprocess.run() u os.system()?
os.fork() hace una copia del proceso actual: el hijo es el mismo programa de Python, con el mismo código y las mismas variables, y sigue ejecutándose desde la línea siguiente al fork()
En cambio, subprocess.run() y os.system() crean un proceso que ejecuta otro programa distinto
subprocess.run() lo lanza directamente y os.system() lo hace a través de una shell
En la tarea hay tres niveles: el padre, el hijo creado con fork() y el proceso mkdir que lanza el hijo. El PID y el PPID que se muestran son los del hijo del fork(), no los de mkdir
2. ¿Qué diferencias observó entre ambas versiones?

- La versión 1 (subprocess.run()) recibe los argumentos en una lista, no usa shell y entrega el código de salida directamente en returncode.
- La versión 2 (os.system()) arma una cadena de texto, usa una shell, necesita shlex.quote() para ser segura y devuelve un estado que hay que decodificar para obtener el código.

6. Conclusión
Con esta práctica se comprobó que os.fork() y os.waitpid() permiten dividir el trabajo entre un proceso padre y un hijo, y que el padre puede esperar al hijo y saber si terminó bien o con errores, en las dos versiones crearon las mismas carpetas y dieron los mismos errores al repetir los nombres, pero subprocess.run() resultó

más segura y fácil de controlar que os.system(), porque no usa shell y entrega el código de salida directamente. También se confirmó que el texto ingresado por el usuario debe validarse antes de usarlo en un comando para evitar la inyección de comandos

7. Anexos:

Anexos Inteligencia Artificial Generativa

Elemento	Descripción del Estudiante
Modelo Utilizado	Google Gemini
Prompt	"Revisa y corrige mi código, y explícame los temas teóricos sobre la creación y sincronización de procesos en Linux. Por favor, guíame con la teoría y los errores puntuales, pero no me ayudes a hacer el laboratorio."
Resultado Obtenido	Explicación conceptual sobre la jerarquía de procesos, el uso adecuado de os.waitpid() para la recolección de estados, y aclaraciones teóricas sobre las diferencias entre subprocess.run() y os.system() al ejecutar comandos del sistema
Validación	Se validó la información teórica comparándola con la documentación oficial y se aplicaron las correcciones de sintaxis sugeridas en los scripts de Python, verificando su funcionamiento en Ubuntu Server

[1] Python Software Foundation, "subprocess — Subprocess management," Python 3 Documentation, 2026. [Online]. Available: https://docs.python.org/3/library/subprocess.html
[2] Python Software Foundation, "os — Miscellaneous operating system interfaces," Python 3 Documentation, 2026. [Online]. Available: https://docs.python.org/3/library/os.html
[3] Linux man-pages project, "fork(2), waitpid(2) y system(3)." [Online]. Available: https://man7.org/linux/man-pages/
[4] Ubuntu Documentation, "Ubuntu Server documentation." [Online]. Available: https://documentation.ubuntu.com/server/
[5] M. Aguirre, “Tarea: creación de carpetas mediante procesos: fork() – subprocess.run() – os.system() – waitpid(),” Sistemas Operativos, Escuela Politécnica Nacional, Quito, Ecuador, 2026